INTERN PROJECT #1 - BIRD'S-EYE VIEW

MedImaging AI

Detect. Explain. Review.

A production-grade chest X-ray and 3D-scan triage system: from raw DICOM files to a radiologist review viewer with explainable AI overlays and a full audit trail.

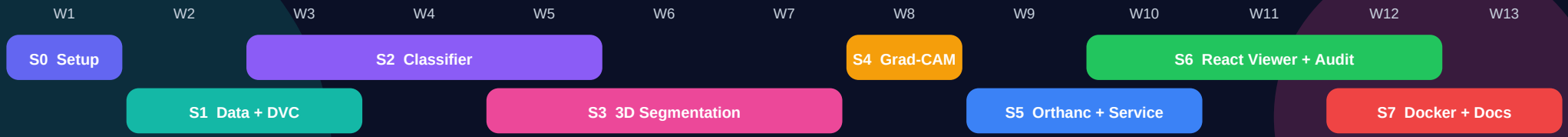
THE SYSTEM IN 5 LAYERS

DATA	DICOM, preprocessing, DVC versioning
MODEL	Transfer-learning classifier, 3D U-Net, Grad-CAM
INTEGRATION	Orthanc server + inference service
APPLICATION	React viewer, Approve/Reject, audit log
DELIVERY	Docker, model card, bias and limits notice

SUCCESS LOOKS LIKE

- [ ] Clone, dvc pull, docker compose up = full working demo
- [ ] Every number reproducible and tracked in MLflow
- [ ] Every radiologist decision lands in the audit log
- [ ] Limits and bias honestly documented
- [ ] Clear 'education only, not clinical use' notice everywhere

13-WEEK ROADMAP



Project Workflow

Raw scan to audited radiologist decision: follow the numbers, left to right, then down and back

MedImaging AI

PHASE A - BUILD AND TRAIN (offline)

1 S0
Data Sources
CheXpert / NIH CXR14 for classification; MSD for segmentation

→

2 S0
DICOM Literacy
pydicom: tags, pixels, window/level, PHI

→

3 S1
Preprocess + Version
Resize, normalize, augment. DVC stages

→

4 S2
Train Models
DenseNet / EfficientNet + 3D U-Net (MONAI), MLflow

→

5 S4
Explain
Grad-CAM heatmap for each prediction

→

6 S5
Package Models
Checkpoints + one clean predict() function

ship model

PHASE B - SERVE AND REVIEW (online, right to left)

7 S5
Orthanc Server
DICOM store with REST API

←

8 S5
Inference Service
FastAPI: pull study, predict, CAM

←

9 S5
Results Store
Database + overlay files + model version

←

10 S6
React Viewer
Scan, AI overlay, confidence score

←

11 S6
Radiologist Review
Approve / Reject (+ reason)

←

12 S6
Audit Log
Append-only: who, what, when, which model

RUNS UNDER EVERYTHING

Git branches + PRs | pytest | YAML configs | MLflow tracking | Structured logs | pre-commit + ruff | Docker (S7)

INTERFACE CONTRACTS - Agree these early and change them rarely. They are the seams between your stages.

1 Data contract
Tensor shape, dtype, value range and label order after preprocessing.

2 Model contract
predict(image) input/output signature, class names, checkpoint + version id.

3 Results contract
JSON schema: study id, scores, overlay path, model version, timestamp.

Tech Stack

Suggested tools per layer and what each one is for. Learn each only as deep as the sprint needs

DATA — Sprints 0-1

- **pydicom**
  read DICOM tags + pixel data
- **SimpleITK / nibabel**
  3D volumes (NIfTI)
- **DVC**
  version data + pipeline stages
- **Albumentations / MONAI**
  augmentation transforms
- **pandas**
  metadata + patient-level splits
- **OpenCV / Pillow**
  resize and image ops

MODEL — Sprints 2-4

- **PyTorch**
  core deep-learning framework
- **timm / torchvision**
  DenseNet-121, EfficientNet-B0
- **MONAI**
  3D U-Net, transforms, sliding window
- **scikit-learn**
  AUROC, F1, calibration
- **pytorch-grad-cam**
  Grad-CAM heatmaps
- **MLflow**
  experiment tracking + registry

INTEGRATION — Sprint 5

- **Orthanc**
  open-source DICOM server (mini PACS)
- **Orthanc REST API**
  /studies /series /instances
- **pyorthanc / requests**
  Python client for Orthanc
- **FastAPI**
  inference service + API
- **SQLite then PostgreSQL**
  results + audit tables
- **Poller / worker**
  detect and process new studies

APPLICATION — Sprint 6

- **React + Vite**
  radiologist viewer app
- **Cornerstone3D**
  render DICOM in the browser
- **Overlay canvas**
  AI mask / heatmap + opacity slider
- **FastAPI endpoints**
  studies, results, decisions
- **Auth (JWT / basic)**
  know who approved what
- **Audit table**
  append-only decision history

DELIVERY — Sprint 7

- **Docker + compose**
  one-command deployment
- **Healthchecks + .env**
  reliability + secrets hygiene
- **Model card**
  intended use, metrics, limits
- **Bias report**
  subgroup metrics, label noise
- **Safety notice**
  'For education only, not clinical use'
- **GitHub Actions**
  lint + test on every PR

DEV HYGIENE — All sprints

- **Git + GitHub**
  branch per task, small PRs
- **pre-commit + ruff**
  format and lint automatically
- **pytest**
  tests for transforms and APIs
- **YAML configs**
  no hardcoded paths
- **Fixed seeds**
  same code, same result
- **Weekly update**
  Mon plan, Fri summary to mentor

Sprint Micro-Tasks | Part 1

Foundation to models: Sprints 0-3 (Weeks 1-7). Tick each box as you go

MedImaging AI

S0 Setup + DICOM Literacy — WEEKS 1

Get a clean workspace and understand your data format.

- [ ] Create repo skeleton, virtual env, pinned dependencies
- [ ] Set up pre-commit with ruff + formatter
- [ ] Write 1-page dataset decision memo (size, labels, license, issues)
- [ ] Note your compute: local GPU, Colab, Kaggle or credits
- [ ] Explore DICOM with pydicom: headers, pixel_array
- [ ] Map Patient > Study > Series > Instance
- [ ] List PHI tags and a de-identification plan

DELIVERABLE **Repo + dataset memo + DICOM note**

S1 Preprocess + DVC — WEEKS 2-3

Make the data pipeline reproducible end to end.

- [ ] Download datasets; configure a DVC remote
- [ ] dvc add raw data; commit the .dvc pointer files
- [ ] Patient-level train / val / test split (no leakage)
- [ ] Resize + normalize (intensity scaling, CT windowing)
- [ ] Augmentations (flip, rotate, contrast): train set only
- [ ] Define dvc.yaml stages + params.yaml
- [ ] Unit-test transforms; visualize a sanity grid

DELIVERABLE **dvc repro works from a clean clone**

S2 Classification Model — WEEKS 3-5

Transfer learning with honest evaluation.

- [ ] Config-driven Dataset + DataLoader
- [ ] Baseline: DenseNet-121 pretrained on ImageNet
- [ ] Handle imbalance: pos_weight / weighted loss
- [ ] Freeze, then fine-tune; LR scheduler + early stopping
- [ ] Log params, curves, per-label AUROC to MLflow
- [ ] Compare EfficientNet-B0; pick winner with reasons
- [ ] Error analysis + calibration check

DELIVERABLE **Evaluation report + best checkpoint**

S3 3D Segmentation (MONAI) — WEEKS 5-7

Train a 3D U-Net you can explain end to end.

- [ ] Start with a small MSD task (e.g. Spleen or Heart)
- [ ] MONAI transforms: spacing, orientation, patch crops
- [ ] UNet + DiceCE loss; mixed precision (AMP)
- [ ] Sliding-window inference on full volumes
- [ ] Log Dice per epoch to MLflow
- [ ] Report Dice + HD95; plot overlays
- [ ] Save checkpoint + standalone inference script

DELIVERABLE **Dice report + overlay gallery**

Sprint Micro-Tasks | Part 2

Explain, integrate, ship: Sprints 4-7 (Weeks 8-13). Tick each box as you go

MedImaging AI

S4 Grad-CAM Explainability — WEEK 8

Show where the model looked, and test that it is telling the truth.

- [ ] Pick the target layer (last DenseNet conv block)
- [ ] Implement Grad-CAM with pytorch-grad-cam
- [ ] Overlay heatmap on the original image; export PNG
- [ ] Sanity check: randomized-weights test
- [ ] Sanity check: heatmap for wrong class vs right class
- [ ] Review 20 cases: lungs, or text markers and devices?
- [ ] Wrap as explain(image) returning heatmap

DELIVERABLE **Overlay gallery + sanity-check notes**

S5 Orthanc + Inference Service — WEEKS 9-10

Studies arrive, predictions appear, nobody clicks run.

- [ ] Run Orthanc in Docker; upload sample studies
- [ ] Explore REST: /studies, /series, /instances
- [ ] Detect new studies (poll changes API)
- [ ] Download study; convert DICOM to tensor
- [ ] Run classifier + Grad-CAM + segmentation
- [ ] Store results (DB + overlay files); make it idempotent
- [ ] Retries, error handling, structured logs
- [ ] End-to-end integration test

DELIVERABLE **Upload to Orthanc, results appear automatically**

S6 React Viewer + Audit Log — WEEKS 10-12

The radiologist's workspace, with accountability built in.

- [ ] FastAPI endpoints: list studies, get results, post decisions
- [ ] React app: study list + scan viewer (Cornerstone3D)
- [ ] Overlay toggle + opacity slider; confidence score
- [ ] Approve / Reject buttons (reason required on reject)
- [ ] Append-only audit table: user, action, time, model version, input hash
- [ ] Basic auth; keep PHI out of logs
- [ ] Persistent 'not for clinical use' banner

DELIVERABLE **Live demo walkthrough**

S7 Docker + Docs + Safety — WEEKS 12-13

Ship it so anyone can run it, and be honest about limits.

- [ ] Dockerfiles: service, viewer (+ Orthanc image)
- [ ] docker compose up starts the whole system
- [ ] Pin versions, healthchecks, env-based secrets
- [ ] Model card: data, metrics, intended use
- [ ] Bias + limits report: demographics, label noise, subgroup metrics, domain shift
- [ ] 'For education only, not clinical use' in README, UI and docs
- [ ] CI: lint + tests on every PR

DELIVERABLE **Clean-clone demo + documentation**

Rules, Risks and Metrics

Pin this next to your editor. Most intern mistakes in this project are on this page

MedImaging AI

GOLDEN RULES

1. **Reproducible: a clean clone gives the same numbers**
2. **Split by PATIENT, never by image**
3. **One branch per task, small PRs**
4. **Track every experiment in MLflow**
5. **No hardcoded paths, no secrets in git**
6. **Justify decisions in writing, with alternatives**
7. **Stuck over 90 minutes? Ask, and bring what you tried**
8. **Thin end-to-end slice first, polish later**

WATCH-OUTS AND FIXES

DATA LEAKAGE
patient-level splits, always

NOISY LABELS
auto-mined from reports: state as a limitation

CLASS IMBALANCE
pos_weight, per-label AUROC, not accuracy

GPU LIMITS
smaller volumes, AMP, small MSD task

SCOPE CREEP
end-to-end slice before improvements

PHI LEAKS
de-identify; no PHI in logs or git

OVERCLAIMING
public-data results are not clinical performance

METRICS + DEFINITION OF DONE

AUROC (per label)	classification ranking quality
PR-AUC / F1	rare findings
Calibration (ECE)	is 0.8 really 80%?
Dice	segmentation overlap
HD95	worst-case boundary error
Subgroup metrics	sex, age, source site: bias check

EVERY SPRINT IS DONE WHEN

- [ ] Code merged via PR, with tests where sensible
- [ ] Reproducible from a clean checkout
- [ ] Metrics + decisions written up
- [ ] No secrets, no hardcoded paths

FOR EDUCATION ONLY - NOT FOR CLINICAL USE

This system is a learning project built on public data. Its outputs must never be used to diagnose, treat or make care decisions. Show this notice in the README, the viewer UI and the model card.